

→ Till here for Test 1.

Date: / /

eg for CI/CD:

→ Auto testing new models

Organizing ML Projects - [Ch2]

Before project starts:

- ML projects should be prioritized due to limited team & compute capacity.
- Org backlog usually exceeds available resources.
- Complexity estimation is uncertain in ML
- Unknown: achievable quality, data volume, feature set

- Clear project goals req before execution
- Team size & resources depend on goal clarity

Q. How do we tell client ⁱⁿ ~~that~~ how much time our project will get completed?

- Literature survey, dataset size
(How other projects were done in past)

Prioritization of ML projects

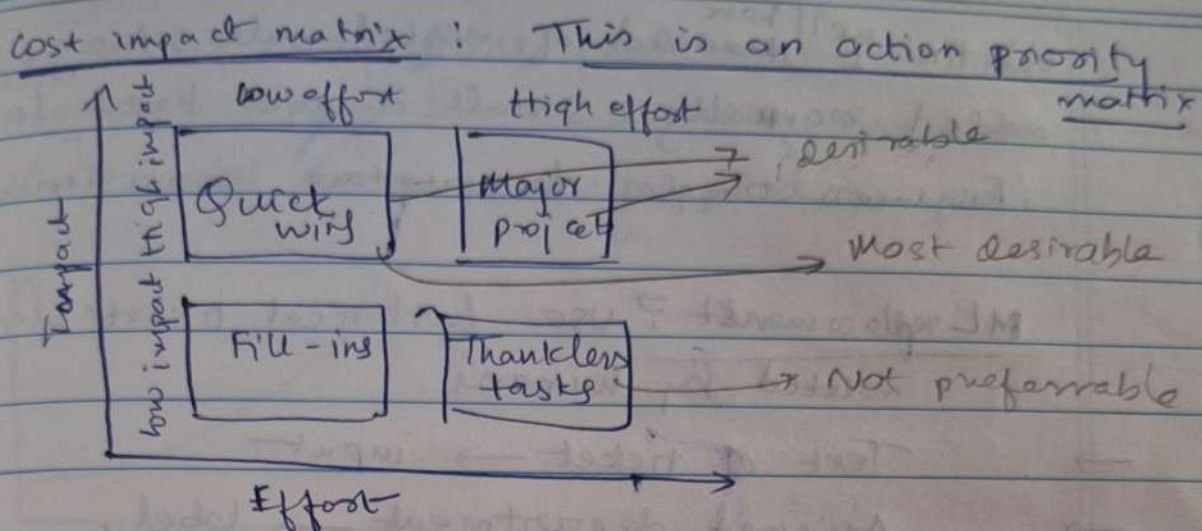
- Prioritization driven by impact & cost.
- Not all ML ideas are worth pursuing.
- High impact, low cost projects preferred
- Requires early technical & business judgement

Impact of ML:

can solve a complex part projects using rule based with many needed rules & exception.

Building & maintaining such systems are

Date: / /



As an owner of company we would want a ML project idea with high impact, low cost. High cost, low impact is least desirable / last priority.

Q: case study for existing rule based systems:

A company receives thousands of customer support tickets daily. Each ticket must be routed to correct department.

Rules look like this:-

- If subject contains "refund" & product = "mobile" & customer type = "premium" → Billing team
- If subject contains "refund" & sentiment = "angry" & order age > 30 days → ~~escalation~~ escalation team.
- If keyword = "not working" & product category = "Software" & OS = "windows" → Tech support
- Else if keyword = "not working" & past ticket exists → Senior Engineers.

Like queries.

& AND



Date: 18/02/2026

Over time we'll have ^{hundreds} of nested rules. Many exceptions added manually. Small changes break ^{existing} logic. Engineers struggle to maintain consistency.

ML replacement → use historical tickets already routed by humans.

- Text of ticket → input
- Assigned department → label.

Why is this a high impact, low cost eg?

- Replaces a complex rule based component
- Historical data exists already → no manual labeling req.
- Reduces eng maintenance effort
- Handles unseen cases better than fixed rules
- Even imperfect predictions are acceptable (tickets can be re-routed)

Cost of ML

Three factors influence cost of a ML

- The difficulty of problem (how complex problem eg:-
- Cost of data (1. Acquisition of data 2. Labeling of data) binary classify vs 100 or 1000 classes
- Need of accuracy (model ok with 70% accuracy vs model ok with 90% accuracy, 90% accuracy model needs more cost bcoz training more req)

• Is an implemented algo or a slw library capable of solving the problem is available?

• Is significant computation power needed to build model or to run in production envt?

• Can data be generated automatically?

• cost of manual annotation of data?

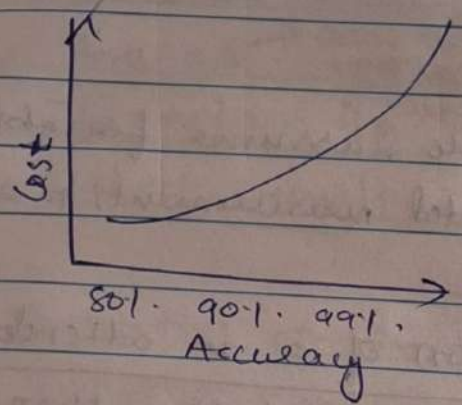
• how many eggs needed?

difficulty of pb

cost of data

need for accuracy

how costly each wrong prediction?
(baseline) what's lowest accuracy level below which the model becomes impractical?



Cost matrix : ^{impact} Action priority matrix (done before ^{previous} pages)

Estimation of complexity of ML project

Best to compare with literature review, no sure way of estimating with surety.

The unknowns → Whether req. quality is attainable in practice.

- How much data will be used to reach the req. quality.
- What features & how many are necessary so that model can learn & generalized sufficient.
- How large model should be (Especially for NN & ensemble arch)
- How long it'll take to train one model (ie, how much time needed to run one experiment) & how many experiments will be required to reach the desired quality.

(Figuring out unknown is through estimates)

Simplifying the problem :

Problem: classifying a set of documents into 1000 topics?

How to simplify?

Focus on 10 topics, label remaining 990 topics as other.
Manually label data for 11 classes (10 real topics + "other")

Date: / /

Execution

Build & solve the model for 11 classes
Measure time at each stage (labeling, training, evaluation)

Inference

If 11 class problem is solvable, assume feasibility for 1000 classes. Use collected measurements to estimate total effort.

Same for face recognition of a ~~an~~ attendance system
See if we can detect the face, then recognize
2 faces atleast, then we can say for many
faces we can recognize

Non-linear progress

- ML project progress is non-linear.
- Prediction ^{error} drops quickly at start.
- Improvement slows down over time.

periods with no visible performance gain.

Time spent on

Adding new features

Using alternate databases / knowledge bases
Labeling more data or outsourcing.

Defining the goal of a ML project :

What a model can do:

- Automate - Triggers actions
Start/stop system activities
- Alert / Prompt - Ask user for confirmation.
Flag suspicious activity
- Organize - Rank or sort items eg:- Google Photos
- Annotate - Highlight relevant parts eg:- news articles
Add contextual labels
- Extract - Named entities (names, companies, loc)
Key info from large files

Relevant items from large collections
eg:- Instagram
Date: / /

- Recommend → Relevant items from large collections
- Classify: Assign i/p to predefined q/s. eg:- Annotation
- Quantity: numeric value expressed in one of these forms. eg:- price, risk score. ML likely not right solution.
- Synthesize → Generate text, images, audio eg:- ChatGPT
- Ans q/s → cross modal or similarity q/s eg:- like chatbots (not simple chatbots tho)
- Transform i/p → Dim reduction, summarization, Augmentation, Translation
- Detect novelty (anomalies)

- Properties of successful model
- What is scientific rigor?
- Solving the right problem
- Two cultures of

if goal doesn't fit in these then it's not suitable for ML problem / release

Structuring a ML Team

Team structure

- Two cultures of ML teams.

Culture 1: Specialists collaborate

Team composition: Data analysts, SW engineers.

Expectations: Data analyst → Handle data well, features, set up experiment, strong ML theory, model design, experimental.

SW engineers → Production-quality code, Scalability systems

Shared requirement → common vocabulary (if diff terms used by diff ppl info can be lost)

Cons: → Integration challenge

→ Scientist solutions may be impractical

Pros: → Deep expertise

→ Faster theoretical solutions

Date 26/02/2026

Culture 2: Hybrid Engineers

Pros: Integration not an issue

- Easier production deployment
- less rewriting of code

Cons: Specialization in domain is missing, Shallow expertise in each area
→ Hard to hire

Team composition: Engineers with: ML skills
S/W eng skills

Behav:

One person owns model + code

Check

on net of chatGPT or TB.

← Case study for culture 1 & 2 using fraud detection system

Members of an ML Team

- ML Engineers → decide what to learn
- Data " → ensures data flows correctly
- Software " → ensures the system works at scale

- Data labeling expert → Ensures truth in the data
- Domain expert → Ensures business relevance
- DevOps expert → Ensures long-term reliability

Members of a ML team

ML engineers / Data Scientists:

- formulates problem as learning task
- select appropriate models
- train them
- evaluate performance against agreed metric

Translate business goals: model ilps, o/p, loss func, eval criteria.
Work is experimental, involves feature eng, model tuning & validation.

Date: / /

→ Accountable for model quality not necessarily production scalability.

Scenario :- Predict student dropout risk in uni defines pb as risk pred task, chooses needed ftrs such as attendance, internal marks, cgpa, distance from home, backlog, hostel status, ^{unactivity,} finance etc & o/p decided as a probability score rather than binary label. They experiment with logistic regression & tree-based model tune hyperparameters & evaluate model to ensure at-risk students are not missed.

Software engineers

focus on turning experimental models into ~~model~~ reliable maintainable systems. Design & implement efficient code, integrate models into existing app & ensure scalability & security. They may not deeply understand ML theory, must understand the ML vocab & constraints to collaborate effectively. To ensure models can run in realworld envts.

Scenario :- Deploying dropout pred model

integrates trained model to university academic portal, ensures pred are generated securely for 1000s of students & handles edge cases such as missing data. They refactor experimental notebook code into a stable service & ensure system remains responsive during peak time.

Data engineers :-

Design & maintain data pipelines that supply ML models. Handle data ingestion, transformation,

Date: / /

Storage & access & using ETL processes. Their responsibility is to ensure that high-quality, consistent, & timely data is available during training & production. w/o robust eng - even best models cannot be deployed or maintained successfully.

Scenario: Prep data for dropout model
builds an automated pipeline that extracts attendance data from ERP system, masks from exam database & activity logs from LMS. They clean & standardize these datasets & ensure the same features are available during both training & live predictions.

Data labeling experts:

creating & maintaining labeled datasets, which are essential for supervised L. Their work includes manual or semi-autom labeling, managing outsourced labeling errors directly affect model performance, this role has a disproportionately large impact on project success, even though it's underestimated.

Scenario: Labeling historical dropout cases.

Labeling experts define what "dropout" means (eg:- not re-registering for two consecutive semesters), manually verify ambiguous cases, & validate label consistency. They also build a simple interface for reviewing student records efficiently & ensure outsourced labelers follow the same criteria.

Date: / /

Domain experts:

provide business & contextual knowledge that cannot be inferred from data alone. Help define what model should predict, which ifs are meaningful, and what errors are unacceptable. Their involvement ensures that model solves right problem, not just technically feasible ones. Domain experts bridge gap b/w raw prod & real business decisions.

Scenario: Academic decision-making

Faculty members explain that a single missed exam should not mark a student as "at risk", but a pattern of declining attendance & internal marks is critical. They also clarify which predictions are actionable early enough to help the student.

DevOps / MLOps Engineers

responsible for deploying, monitoring & maintaining ML models in production. They automate model serving, handle retraining, manage retraining pipelines & monitor data drift & performance degradation. In many orgs, they ensure models remain reliable & reproducible over time. In smaller teams, this responsibility may be shared with ML engs.

Scenario: Monitoring deployed model

sets up monitoring to detect when student behavior patterns change (eg: - due to online semester), schedules periodic retraining & tracks model performance over time. Ensures ^{that} if model fails or degrades, alerts are raised automatically.

Date: 3/03/2026

Machine learning projects: why they fail.

74-87 1. of ML learning projects fail.

Causes: (could be combination of these too)

1. Strategic & organizational causes

- (1) Lack of support by leadership: Leaders may view the high uncertainty of ML experiments typical of scientific work as a "lack of progress" & lose interest when results are not immediate.
- (2) The highest paid person's opinion: can often override data-driven decision-making, leading to a focus on the wrong problems.
- (3) Lack of business Alignment: Many projects start w/o a clear technical understanding of the business objective. Scientists may focus on technical metrics like accuracy, but if these do not translate ~~results~~ into increased click-through rates or revenue, the business may skelne the result.

Talent & Collaboration traps

- Lack of experienced talent: Much of available workforce has superficial expertise obtained from toy datasets in a classroom context & lacks experience with the entire machine learning project life cycle.
- Siloed organizations: Friction occurs when data assets reside in diff departments with diff ownership & security constraints. A lack of trust & collaboration b/w data engineering, data science & ~~anal~~ DevOps team can cause a project to be frozen for long periods.

Engineering & operational failures

- Lack of devops foundation: w/o automated testing (CI) & deployment, it's impossible to meet full potential of a ML system.
- Missing data infrastructure: model may work in research env but the production infra often fails to generate inputs for the model fast enough to be useful.
- Lack of operational experience excellence: Real world ML systems incur massive ongoing maintenance costs that teams fail to account for.
- Silent failures: Unlike

Data challenges

Data labeling Bottleneck: 76% of teams label data on their own, leading to significant time spent by highly skilled scientists on manual tasks. If labeling is outsourced w/o quality validation, the resulting data can be of low quality or entirely wrong.



Date: / /

Bias & ethical consequences. Models trained on biased data produce biased results. If an engineer focuses 100% on engagement & profit while ignoring ethical & unintended consequences, the system may create harmful "filter bubbles" or extremist content recs that eventually lead to project failure or forced decommissioning.

Case study - Fintech startup

A fintech startup is developing a ML model to predict customer credit default risk. The project team includes data scientists, risk analysts, software eng, product lead & senior exec. The data science team builds a gradient boosting model trained on historical repayment data, transaction patterns, credit history, & behavioral features. Offline eval shows strong ROC-AUC performance & improved recall for high-risk customers. A pilot deployment indicates a projected 8-12% reduction in default losses compared to existing scoring system.

During final approval meeting, the CRO who has significant authority over risk decisions, argues that traditional rule-based credit scoring is more "transparent" based on intuition. The team didn't counterargue on fear of authority.

Competitors profited by deploying it early, ~~the~~ the startup lagged behind due to delay in deployment & had losses.

1. What is the phenomenon we are seeing? ^(HIPPO)
2. Identify the phenomenon.
3. Explain how it influences decision-making process in credit risk model deployment.
 (CFO overrode expert's advice in favor of intuition, overriding the data science team's recommendation)

→ suboptimal model selection, financial loss due to default rates & erosion of competitive advantage
innovation & creates long-term organizational inefficiency

- A Analyze the potential risks of such decisions dynamics in data-driven financial systems.
- B Propose structures mechanism that the organization could impl to prevent similar outcomes in future
- ans → Predifine success m

The Hippo effect (Highest Paid Persons Opinion)

→ A decision-making bias where senior authority overrides empirical evidence.

→ Common in hierarchical organization

How it manifest?

Executive intuition replaces experimental validation.

Data-backed recommendations are dismissed

Team members hesitate to challenge authority

Decision is made w/o A/B testing or metrics

review (w/o rigorous testing)

Warning signs:

How do you actually prevent it?

- Predifine success metrics & approval thresholds
- Mandate experiment-based validation
- Use structural review committees
- Encourage formal dissent mechanisms
- Document decisions & supporting evidence.

Position till midterm